

COMPLETE WORKING RECORD

A NEXT GENT Playbook

Everything from one long working session: the thesis, the machine, the architecture and why, the products, the economics, the go-to-market, the brand, the open decisions, and what to do first.

01

The thesis

You are selling churn absorption. Red Hat's subscription is not access to Linux — it's a promise to absorb upstream churn so an enterprise never has to. Customers renew because upstream never stops moving.

Your upstream is the user interfaces of the apps a small business runs on. Toast redesigns the hours screen. Square moves a button. Google Business ships a new layout. Every DIY automation in America breaks silently and the owner finds out from an angry customer at a locked door.

Yours breaks too — for about six hours.

```
1 a vendor ships a UI change
2 post-action verification fails across the fleet — you know first
3 you re-map that one flow, once
4 signed, pushed to every customer overnight
5 nobody noticed anything happened
```

The flow library is **shared**. Mapping Toast's hours screen costs the same whether you have four customers or four hundred. Marginal cost of the 401st Toast customer is near zero; value to them is identical.

App UIs change far more often than kernels do. That makes this a stronger recurring business than the one it's modeled on.

The fourteen-year insight

The hard part of a directory or a search engine was never the index. It was the data. What changed isn't storage — it's that every business now *receives* its own structured operational data by email, and runs apps that can be operated. **The acquisition mechanism is what was missing.**

The 2013 framing still holds, unchanged, and it's the pitch:

"Nobody cares about 1.8 trillion search results. They have a faucet leak. They need a plumber now."

02

What you're selling

One foundation, many sellable surfaces. A customer can start with a QR menu and end up on the full operating environment without you rebuilding anything.

PLATFORM

A NEXT GENT

The operating environment: identity, workspaces, Constitution, permissions, agents, capabilities, apps, data, automations, devices, proof history. The model underneath is replaceable.

HARDWARE

Ghost

The physical node. Optional — a deployment of A NEXT GENT, not the platform itself. Version one: *the Ghost of Lisa*.

DATA

CyberCheck

Canonical structured business truth with provenance and controlled public/private exposure.

DATA

Quantum Warehouse

Source crosswalks, normalization, history, reconciliation. How you know, where it came from, which source is authoritative.

ECOSYSTEM

VAPPS + App Store

Capability packages. Developers build once against stable capabilities; any business grants access in one tap. 0% to developers.

DIRECTORY

GCR

Local discovery powered by verified, consented, item-level business truth. The consumer side of the same data plane.

Four businesses on one foundation

- **Software** — apps, assistants, websites, booking, menus, reviews, loyalty
- **Hardware / managed computing** — Ghost, Android execution, voice stations, displays
- **Platform / developer** — VAPPS, App Store, SDK, marketplace, APIs, certification
- **Data / network** — CyberCheck, Quantum Warehouse, GCR, partner and chamber networks, agent-accessible business truth

Same identity, same workspace, same Constitution, same capability registry, same execution layer, same verification, same ledger. Four revenue lines, one backend.

03

The machine, end to end

Four machines

WHERE	WHAT LIVES THERE
Workstation	Flow mapping and factory. Maestro Studio, appium-inspector, trailblaze, browser-use, syft, grype, Tekton. Never ships.
Cloud	Quay, Pulp, Katello, Foreman, flightctl, Candlepin, rekor, Zitadel, Postgres, Lago.
Box	bootc OS, Podman, zot mirror, llama.cpp, litellm, ContextForge, uiautomator2, Playwright, Wyoming, SpiceDB, node agent.
Phone	\$40 Android, their SIM, their logins, TextBee, accessibility service. You don't ship it.

One request, all the way through

```
owner texts "post tonight's special"
  ↓ TextBee – phone, real SIM
  ↓ litellm → local model → capability + arguments
  ↓ Zitadel (who) → SpiceDB (may they)
  ↓ risk tier → medium → run and notify
  ↓ execution-router: Facebook has no usable API here → Android
  ↓ uiautomator2 replays the AppMap – fingerprint checked first
  ↓ Playwright reads the public page back – different path than wrote
  ↓ expected == observed → screenshot + post ID → Proof Ledger
  ↓ SMS back: "Posted ✓ verified – ACT-20260913-00814"
```

Every layer is a separate container. Swap llama.cpp for vLLM, uiautomator2 for Artemis, SpiceDB for OpenFGA – nothing above or below changes.

The four ingestion surfaces

- **Email** — every vendor already sends the business daily summaries, receipts and review notifications. Universal, free, and they can't turn it off without breaking their own product.
- **QR menu** — your own sensor. Scan time, table, dwell, what they browsed. No POS generates this.

- **Remote control** — reading state off apps that have no feed.
- **Owner text-back** — the daily toggle. "Catch of the day?" "Still have grouper? Y/N."

NOTE That text thread is the heartbeat. Freshness is what every downstream surface is worth. If they stop replying, the menu, the website, the directory and the phone agent all decay — and stale data poisons faster than no data.

Session reconstruction

```
19:04  scan      table 12
19:04  browse     apps → entrées, 90s on seafood
19:11  order       calamari, shrimp scampi, 2 bushwhackers
20:02  checkout    $94.20
20:15  text        "how was the scampi?"
20:19  review      4.9 — verified, item-level
```

Four sources joined into one **visit**. Toast doesn't know about the scan. Google doesn't know what they ate. You know all of it and the business owns it.

04

Architecture decisions and why

Image-based OS, not package-based

Where the system's state lives — and who assembles it — decides your support cost and what you can promise a buyer.

	REGULAR LINUX	CURATED MUTABLE	IMAGE-BASED
State assembled	on the device	on the device	in your CI
Machines identical	no	roughly	byte-identical
Update unit	~2,000 packages	packages + migrations	1 signed image
Can fail halfway	yes	yes	no
Rollback	none	manual, partial	automatic, invisible
Migration debt	n/a	grows forever	zero
Scales with	machines × configs × versions × hardware		number of SKUs
Subscription gating	hard — whole repo	hard — whole repo	trivial — one pull
10-year promise	sort of	no — rolling release	yes

The row that matters most is the second-to-last. Support cost in the first two columns grows combinatorially; in the third it grows linearly with SKU count. That's the difference between hiring your way to 100,000 customers and drowning at 5,000.

The two rules the runner never bends

- RULE 1

Fingerprint before acting. One or two elements must be on screen before the first tap. If they're not, stop and report DRIFTED. An app redesign fails loudly on the first machine instead of quietly changing the wrong field on four hundred.
- RULE 2

Read the result back, on a different path than you wrote it. Wrote via Android, verify via browser. If the verifier is the executor, you've built self-certification with extra steps. No verify block, no OK.

Publish, don't push

Never build a pipe into the customer's machine. You publish a signed image to a registry; the device polls, checks the signature, and decides. **You cannot be compelled to do something you have no**

mechanism to do. No government, plaintiff, or attacker with your credentials can target one customer, because targeting doesn't exist in the architecture.

Prove it with a **public transparency log** (rekor). Devices refuse anything not publicly logged, so a targeted build would have to be published to the world first. That turns "trust us" into "you don't have to."

BETTER USE Put the transparency log on **constitution versions and AppMap promotions**, not just OS images. Those are the claims that actually need third-party verification.

Take money without taking identity

Billing knows who paid. The update service knows only that *a* valid subscriber asked. Blind-signed tokens (Privacy Pass — deployed in production by Cloudflare and Apple) make the link mathematically impossible, even for you.

"I don't hold your data" stops being a policy that changes when a board changes, and becomes a property of the system. You couldn't produce a user list under subpoena because you don't have one.

The agent is deliberately dumb

Intent → column → answer. No generative step means no hallucinated price. Same strict schema for every business in a vertical means no per-customer prompts, no per-customer tuning, no per-customer regressions — and no training.

The schema is the product. The model is a lookup. Anyone can buy the voice layer for five cents a minute. Nobody can buy columns that are correct right now.

- **Never guess.** Off-schema means a clean handoff plus a text to the owner. Never a plausible-sounding answer.
- **Log every unknown.** After 500 calls across 20 businesses that log *is* the complete schema for that vertical, ranked by real demand. Free roadmap.
- **Show freshness.** Every fact carries an observation timestamp. Stale beyond threshold, it degrades to "let me check."
- **Answer fast, execute slow.** Voice needs sub-800ms. If booking needs remote control (10–20s), confirm immediately, hold the slot against your own inventory, complete in the background.

Local models: the right shape

Disk isn't the constraint — RAM and load time are. An 8B model at 4-bit is ~5GB and 2–8 seconds to load. Eight seconds of "loading the cooking model" is a dead product.

TIER	WHAT'S THERE	SWAP COST
Always resident	1–3B router + embedding model	never unloads
Hot	one strong 8–30B base, permanently in RAM	never unloads
Instant	LoRA adapters — legal, medical, code, cooking, finance	~200ms
On demand	genuinely distinct models — vision, speech, big coding	5–20s, honest UI

50 specializations as adapters cost ~5GB instead of ~400GB, and swap in 200ms instead of 8 seconds. Ship 150–250GB of curated models and adapters, not 1TB.

Hardware implication: unified memory is the only affordable way to give a GPU 64–128GB. AMD Strix Halo (Ryzen AI Max+) with in-tree amdgpu keeps the single-image OS clean — that's an architecture decision, not a benchmark one. Ignore NPU TOPS claims; NPUs are near-useless for LLM inference today.

05

The six unsolved problems

Specified across the documents but not solved, and no repo in the 643-item census solves them for you. These are the ones that will break the build.

1. Capabilities defined by postcondition, not action

`menu.update_price` runs via API on the website, a browser map on Google, an Android route on Toast. If the capability is a *sequence of steps*, you get three unrelated implementations and no way to say whether they did the same thing.

A capability is a state assertion plus a set of executors that attempt to bring it about.


```

capability: menu.update_price
postcondition:
  subject:      menu_item{id}
  predicate:    price == $18.99
  surfaces:     [website, google, facebook, qr_menu]
  observable_by: [api.read, browser.read, android.read]

```

Executors become interchangeable by construction and verification is derived from the contract instead of written per-executor. This is the change that makes the execution-router coherent rather than a switch statement.

2. Verification has three states, and must run on a different executor

- **Propagation delay.** Google Business doesn't reflect a change instantly. Read back immediately and you record a false FAILED. Every capability needs a **settle window** and retry-until-stable before it asserts anything.
- **Partial application.** Price landed on 2 of 4 surfaces. Not VERIFIED, not FAILED.
- **Unsafe re-read.** Some actions can't be probed without cost or side effect.

Result type is VERIFIED | UNVERIFIABLE_YET | CONTRADICTED | PARTIAL, and UNVERIFIABLE_YET is a schedulable state, not an error.

STRUCTURAL The verifier must not be the executor. If the browser map both writes and reads the Facebook page, that's self-certification with extra steps. Write via Android, verify via browser. The independence is the proof.

3. Failure is the unbuilt half

Every document is about proving success. Operationally the money is in what happens on CONTRADICTED. Three things missing from the capability spec:

- **Idempotency keys.** A retry must not double-post, double-charge or double-book. Every ActionSpec carries one; every executor honors it. Without this, retry is a liability.
- **Compensating actions.** A price change applied to 2 of 4 destinations is *worse* than one that failed cleanly — the business is inconsistent and nobody knows. Each capability declares its compensation: roll the two back, or drive the other two forward. Pick one per capability, in the manifest.

- **Escalation thresholds.** Which failure, after how many executor attempts, becomes an SMS rather than another retry. A policy value, not a constant.

4. AppMap versioning — the moat's hard problem

scrcpy and Maestro are commodity. The accumulated map — routes, fingerprints, recovery states, reliability stats per app per capability — can't be bought, and improves for every customer when one breaks. The unsolved part is binding.

- **Pre-flight fingerprint.** Before acting, hash the screen's structural signature and compare to what the map expects. Mismatch means stop and escalate to the AI executor — *not* proceed and hope. Acting on an unrecognized screen is how you change the wrong field.
- **app_version × map_version as the registry key**, not app → map.
- **A promotion pipeline with a gate.** AI discovers the new route → replays in sandbox against a known state → canary on a small slice → promoted to deterministic. The gate criteria decide whether a bad map reaches 400 customers.

Facebook / publish_post / browser task pattern					
success	9,821 / 9,847	verified	99.72%	4.1s	\$0.002
Facebook / publish_post / AI GUI agent					
success	903 / 1,000	verified	90.3%	17.0s	\$0.080

When deterministic reliability falls, route to the AI executor, learn the changed workflow, test it, promote a repaired deterministic pattern. Those numbers seed the execution-router's scoring — use android_world to generate them rather than guessing.

5. CyberCheck needs a conflict resolution policy

Keeping conflicting observations instead of overwriting is the best idea in the documents. But confidence = 0.95 has to come from somewhere.

Observed		
Google	Friday close = 9 PM	observed 5:34 PM
Facebook	Friday close = 10 PM	observed 5:36 PM
Website	Friday close = 10 PM	observed 5:37 PM
Canonical		
Friday close = 10 PM confidence 0.95 last verified 5:40 PM		

- **Per-field source ordering.** Menu prices: POS > website > Google > Facebook. Hours: owner declaration beats everything. A table you write once per field class.
- **Staleness decay.** Forty minutes old outranks six days old; both decay toward unknown.
- **Escalation rule.** Two high-trust sources disagreeing past a threshold is an owner question, not an automatic resolution. Silently picking a winner is how you confidently publish the wrong price.

6. Put the transparency log where it matters

Signed OS images are table stakes. The externally-verifiable claims that actually matter are **which constitution version authorized an action** — with proof that version existed then and wasn't written retroactively — and **which AppMap version was promoted, when, on what evidence.**

That turns the Proof Ledger from something you assert into something a third party can check. It also answers the constitution service's in-flight problem: an action binds to a constitution version at ActionSpec creation and carries it to completion even if the owner amends mid-task, and the amendment is logged with its own timestamp so the sequence is provable.

The policy decision ladder

ALLOW	standing rules clearly permit – continue
ALLOW_WITH_CONSTRAINTS	only this resource, amount, recipient, window
REQUIRE_APPROVAL	human, task-bound, expiring, single-use
DENY	never, regardless of who asks

- No agent may expand its own authority. Possession of a tool, model, credential, device or network connection does not imply permission to use it.
- A model recommendation is not an authorization decision.
- Raw credentials are never implied by permission to perform a capability.
- Agents request by scope and purpose, never reaching directly into stores.
- Execution success is not proof of outcome — postcondition verification stays separate.
- Human approval is narrow, task-bound, expiring and single-use unless the owner explicitly changes the standing rule.

- Even a full-authority agent routes privileged requests through the gateway, so access, actions, data released and resulting proof are documented. Full authority means policy auto-allows within scope — not bypassing the audit path.

06

The data plane

The gate is closed at the vendor. It's wide open at the business.

Toast won't give you an API. But Toast already sends that restaurant a daily summary, a receipt for every order, and a notification for every review — into an inbox the business owns. Nobody can block that without breaking their own product.

Collect at the business, not at the vendor. It works for every vendor at once, including the ones that don't exist yet.

A consented data plane, federated

Not a data aggregator. The business is the holder; it grants scoped access and revokes it. You never own a central pile — which is why businesses say yes, and why "decentralized" is accurate rather than marketing.

ingestion	Tika / Docling parse, langextract structures
mapping	Quantum Warehouse — which field means what, where from
truth	CyberCheck — canonical, with provenance
grant	capability-registry + policy-engine — one-tap approve
wire	MCP and A2A

Developers get answers, never rows

```

developer app → "offer 15% to anyone who ordered scampi in 90 days"
               ↓ A2A
business agent → runs locally against local data
               ↓
returns        → "sent to 312"

```

They never learn who, never learn numbers, never learn what else was ordered. Five things enforce it:

1. **Capabilities defined without data in the signature.** `offer.send_to_cohort` returns a count. `customers.list` doesn't exist.
2. **Developer logic runs in WASM with no network egress** — Extism + wasmtime. Only approved host functions. The code physically cannot phone home.
3. **Scoped, revocable, time-bounded grants** — per purpose, visible in the owner UI, one tap to kill.
4. **A2A as the hop** — capability invocation, not data transfer.
5. **Every access in the Proof Ledger** — which developer, which capability, what came back.

THE LEAK The return value is the attack surface. Enough narrow questions reconstructs the dataset — "how many ordered scampi," filtered a thousand ways, rebuilds the customer list one count at a time, and every query looks harmless. Fix by shaping returns: minimum cohort size (10–25), rate and query budget per grant, aggregates-only for analytics scopes, and enumeration-pattern detection in the ledger. Solved in principle, almost never implemented — doing it properly is a real differentiator.

Machine-readable or invisible

A restaurant's menu as a PDF is invisible to AI. Emit four surfaces on every business page:

- **Schema.org JSON-LD** — Restaurant, Menu, MenuItem, Offer, AggregateRating, OpeningHoursSpecification, Reservation
- **/well-known/agent-card.json** — A2A discovery
- **An MCP endpoint** — for agents that ask rather than scrape
- **llms.txt** — costs nothing

Per-dish AggregateRating backed by verified reviews is the part nobody can fake.

Agent analytics — a product nobody sells

This month, AI agents asked about your business 1,247 times

hours / open now	412	
menu items	338	
availability tonight	201	
gluten-free options	94	← no data here
parking	67	← no data here
dog friendly	41	← no data here

Google Analytics says someone visited. This says what they wanted to know. For the owner that's the renewal. For you, aggregated across 200 businesses, it's a demand-ranked schema roadmap.

Public JSON-LD is free and anonymous. The richer MCP/A2A surface requires identification — live availability, item ratings, the ability to book. Good trade for the agent, and the agents that matter end up identified.

07

Products and verticals

Real-time availability — the biggest one

A charter doing \$400K/yr through FareHarbor at 20% pays **\$80,000 a year in commission**. They pay it because their own website can't show real availability. Shift 30% to direct and they save \$24,000. Your \$499/mo is \$6,000.

SALES CALL 4:1 return on one line item, provable from their own statements. Not "saves you time" — "here's twenty-four thousand dollars."

declared capacity	6 jet skis	
FareHarbor email	2 booked 10:00-12:00	→ 4 left
Booking.com email	1 booked 10:30-12:30	→ 3 left
cancellation email	1 released	→ 4 left

FareHarbor doesn't know about the Booking.com reservation. You're the only thing that sees all channels at once.

- **Email lag** — confirmations arrive 30s–2min late. Hold a safety slot; show 4 when you have 5.
- **Walk-ins generate no email** — one-tap decrement on the owner's phone or the wall screen. Design it before launch.
- **Template churn** — fingerprint the format, fail loudly, re-map once, push to everyone.
- **Drift** — reconcile by reading real platform state through the remote control on a schedule. Your verification layer applied to inventory.

Verified item-level reviews

The box reads the POS. It knows who, when, and exactly what. The review is tied to a real receipt with real line items — not "4 stars, good food."

Chicken Alfredo	4.7	• 312 verified orders
Shrimp Scampi	4.9	• 188 verified orders
Calamari	4.2	• 241 verified orders
Bushwhacker	4.8	• 906 verified orders

That's a dataset, not a feature. It feeds the QR menu, the public page, the owner's margin decisions, the directory, and recommendations. Accumulated across a region it's unrebuildable.

The ask writes itself: *"Hey Bob — how was the chicken alfredo?"* gets answered. *"How was your visit?"* doesn't.

- **Host it first.** Reviews live in CyberCheck and on your surfaces. Syndication out is downstream and optional. Build it the other way and you're a review-request tool that owns nothing.
- **Never comp for a review.** The instant there's an incentive the ratings are worth less, and being trustworthy is the entire value. A real 4.2 beats a fake 4.8.
- **Publish only on completed visits** — verified by POS ticket or QR scan.

The phone assistant

Every AI phone agent on the market fails on **data, not voice**. They're grounded on a static menu PDF and a hardcoded hours string from setup day, so they quote March's price and offer a booked slot.

The business turns it off in three weeks.

Voice is \$0.05/minute and commodity. You own the half nobody has: live availability, current prices from the POS, today's hours, what's 86'd.

A restaurant misses 20–40% of calls during rush. An answering service is \$200–500/mo and can only take a message. Yours books revenue.

Remote phone access

Most small business owners carry an iPhone and physically cannot run Android-only business apps. The handset lives in the cradle at the shop; they see and touch it from their iPhone anywhere. **That's a reason to buy the box before any AI does anything.**

ws-scrpcy	browser front-end, screen + touch, works in iOS Safari
scrcpy + adb	over the network instead of USB
headscale	phone app → box → handset, point-to-point encrypted

Same connection the agent uses, so human takeover is free — and what they do by hand becomes a new AppMap.

Multi-account ceiling: Android work profile or OEM app cloning handles two instances comfortably. Five separate Airbnb logins needs more handsets — design the cradle for two or three from the start.

- Device-bound auth, enrolled once, keyed to the box
- Nothing on your servers — the relay is the mesh
- Every session in the ledger, visible to the owner
- A physical kill switch on the box

SMS from their own SIM

Every SMS marketing platform's margin is the per-message markup. Yours is zero — it's the customer's SIM on their plan. And the box already has the list, because it's reading bookings, POS, reviews and calls.

"No per-message fees, ever" is a sentence no competitor can say.

WATCH Carriers throttle bulk sending from a person-to-person line — low hundreds per day before deliverability degrades. Split it: own SIM for conversational, 1:1, approvals and small batches; a registered A2P route behind the same interface for real blasts. Test the actual daily ceiling before putting a number in marketing.

Artist song requests

QR on the stage → landing page → request + tip → confirmation email parsed → identity tied to request. Zero new architecture.

Fifty artists playing 4–6 nights a week to 100–300 people is roughly **30,000 impressions a week**, in venues, with people holding their phones in spending mode. Give the artist product away entirely — they're the channel, not the revenue.

FLYWHEEL The QR stands in a bar every Friday. The owner watches it work and asks the musician what it is. **Sign artists to sell bars.**

Don't process the tips. Let them use their own Venmo or Cash App — free between people, "you keep 100% of your tips," and the confirmation email still gives you the tie-together. Same mechanism as everything else.

Three consumer queries nobody can answer

- "Who has crab legs nearby?" — menu data is in PDFs, behind logins, or stale on Google
- "Who's playing live tonight?" — Bandsintown and Facebook events are stale and incomplete
- "Two bed two bath at Silver Shells, these dates" — nobody lets you search by *complex*, which is how beach markets actually search

Same machine answers all three. The third is the highest value — the transaction behind it is \$2,500, not \$90.

Cross-business recommendations

This person had the grouper at A, the scampi at B, rated both — recommend the snapper at C. Yelp has reviews but not verified purchases. Google knows you visited but not what you ordered.

Useless until a market is dense. Twenty restaurants in one town works; two hundred across three states is noise.

08

Economics and pricing

Price against what you replace, not against a laptop. Your comps are Yext (\$199–999/yr), Birdeye / Podium / SOCi (\$250–500/mo), a part-time VA (\$1,500–3,000/mo), an answering service (\$200–500/mo). Those manage listings. You execute.

LINE	PRICE	NOTE
Software	\$249 / mo	Their hardware, their phone. Core flows, approval queue, SMS interface.
Ghost	\$499 / mo	Box + handset, \$0 down, 24-month term. Unlimited flows, voice, phone line, priority support.
Multi-location	+\$299 / mo	Each additional location.
Regulated	\$799 / mo	Clinics, firms. Retention controls, compliance logging, SLA.
Onboarding	\$1,500–3,000	One-time. Connect accounts, map flows, configure, public pages. Filters tire-kickers.
Custom AppMap	\$500–2,000	Weird vertical app. You map it and maintain it.
Developers	0%	Say it loudly. Apple takes 30%. It's marketing.

Bundle the hardware. \$2,495 up front gets "let me think about it." \$499/mo gets a yes. Toast, ADT and every successful small-business hardware play does it this way.

PRICING Start at \$499, not \$249. Same sales call, same support load. If nobody pushes back you priced too low, and you can always discount the first ten for proof. You can't raise on people you signed cheap.

Transactions — 0% direct, a cut on what you bring

BOOKING CAME FROM	YOU TAKE
Their own website	0% — they just pay Stripe 2.9% + \$0.30
Your directory	X% — you keep it
Partner directory	X% — split with them
Revenue you created	10% — refills, last-minute, cancellations filled
FareHarbor / Booking.com	nothing. Don't chase it, don't invoice.

"I don't take a cut of your existing business. Only what I bring you." Structurally aligned instead of taxing them — and the partner split makes other directories *want* to carry your data, which funds distribution.

NEVER A cut you have to invoice for is the worst of both worlds. If the booking happened on FareHarbor, the money lands in the business's account and you're sending invoices to a charter captain in February. Own the rail for bookings you create; take nothing on the rest.

Payment facilitation — the real long-term line

Standard model. Toast, Square, Shopify Payments all negotiate wholesale and resell blended. Stripe Connect supports it natively — `application_fee_amount` is your spread.

	YOUR COST	YOU CHARGE	SPREAD
Card present	~1.8-2.1%	2.9%	0.8-1.1%
Online booking	~2.4-2.7%	2.9% + \$0.30	0.2-0.5%

1.9% all-in isn't reachable for card-not-present. Interchange alone is 1.8–2.3% + \$0.10 and nobody negotiates below it. What you get at volume is interchange-plus, roughly IC + 0.4–0.6% + \$0.10–0.20.

- ~\$1M/yr — a conversation, not a rate
- ~\$12M/yr (\$1M/mo) — real interchange-plus on the table
- \$50M+/yr — genuine leverage, custom terms
- \$200M+/yr — direct processor or becoming a payfac is worth the compliance burden

HIDDEN COST Dispute liability. With destination charges the **platform** is liable by default, plus negative balances when a business closes holding deposits. Budget 0.1–0.3% of volume for disputes and reserves, and set the liability structure deliberately. That's what turns a 1% spread into 0.7%.

Why 0% is the highest-leverage price in the model

Shopify charges 0% transaction fee *if* you use Shopify Payments. The free tier isn't generosity — it's volume acquisition, and the processing spread is the business.

Free on direct → they consolidate onto your rail → you get 100% of their volume → volume buys the rate → the rate is the revenue. Every competitor charging a platform fee on direct bookings leaves that consolidation on the table.

Target high-ticket for volume

CUSTOMER TYPE	AVG TICKET	ANNUAL VOLUME EACH
Vacation rental manager (40 units)	\$2,500/wk	\$2–4M
Hotel / condo property	\$200–400/night	\$1–5M
Charter fleet (6 boats)	\$800/trip	\$800K–1.5M
Marina	varies	\$1–3M
Single charter	\$800	\$200–300K
Restaurant	\$60–90	\$1–3M, tiny tickets

Twenty vacation rental managers is \$60M/year in volume. Twenty sales calls to serious negotiating leverage, versus 500 restaurants.

Split the targeting: property managers and charter fleets for payment volume, restaurants for data density and the directory. Different sales motions — don't run them as one. Watch seasonality (Gulf Coast can be 5:1 summer to January — negotiate on trailing-twelve after peak) and concentration (no vertical past ~35% of volume).

Loyalty points

A single-business loyalty card is worthless to someone who visits once. **Regional points** — earn on a charter, spend at a restaurant — is something no individual business and no OTA can offer, and it's the first thing that gives a tourist a reason to use your directory instead of Booking.com.

- **Clearing** — the business giving the discount isn't the one that earned it. You're already in the middle of the transactions. Decide up front: pool buy-in, or earning business funds the liability at issue.
- **Terms and expiry from day one.** Unredeemed points are a liability on paper and revenue in practice. Retrofitting terms is a fight you don't want.
- Keep v1 a points ledger and a redemption code. Don't build a currency.

Referrals — the version nobody can fake

Affiliate content is drowning in AI slop. A creator video tied to a verified booking *and a verified completed visit* can't be manufactured. They can fake the face; they can't fake the receipt.

creator link → booking → visit completed → verified → payout

KEY Pay on verified completion, not on booking. Every affiliate network pays on a conversion they can't see. You're standing at the end of the transaction. That kills fake bookings, cancel-after-payout, and owner self-dealing in one move — and no competitor can claim it.

Decide attribution before launch. Ambiguous case — found on your directory, booked direct two days later — **default to 0%**. Costs a little revenue, buys the thing the model runs on. Write the policy down and show it to them.

White label — the GoHighLevel model

Don't sell 100,000 businesses. Sell agencies with 20 clients each. They do sales, onboarding, support and customization; you run the platform. **And it closes the biggest hole in the direct model — support cost at scale.**

Agency pays you	\$199-249 / box / mo wholesale
Agency sells at	\$499-699
Agency keeps	\$250-450 / mo per client, plus install and
You get	predictable revenue, zero support load, zero CAC

An agency with 40 clients makes \$12-18K/month off your platform. Your trained rental-ops reps are exactly the people who become those agencies. And unlike a GHL agency they get a service business too — installs, phone setup, flow mapping — which makes them stickier.

Revenue math

- 100 customers × \$499 = **\$599K ARR** plus ~\$200K onboarding in year one
- 250 = **\$1.5M**
- 1,000 = **\$6M**
- 200 businesses × \$16M/mo processed at 0.7% net = **~\$1.3M/yr**, growing with their revenue rather than your headcount

METRIC **Flows per customer** is your only real retention number. Two flows and they churn. Fifteen and they've moved their operations onto your box and they're not leaving. Instrument it from customer one.

Go to market

The email forward is the entry point, not the box

"Here's an email address. Forward your booking confirmations to it. That's it."

Five minutes, no hardware, no install, no phone, no card. With just that you deliver real-time availability, verified bookings, a public page and widget that work, and the beginning of their structured data.

That's the free tier. The box is the upsell for *execution*. It collapses CAC to nearly nothing, kills the developer chicken-and-egg, and means your installed base can be ten times your paying base — every forwarding business in the directory, feeding the data plane, already half sold.

Scale by vertical, not by geography

The schema is the unit of work, not the map. A rental manager in Destin, Gulf Shores and Panama City are the same schema, same flows, same emails. Once the vertical is built, customer 2 through 200 cost nothing new regardless of where they are.

Density still matters for the consumer directory and the developer market — a developer needs to see an addressable market — but it arrives as a side effect of going deep in one vertical first.

Channels, ranked

1. **Multi-unit operators.** A franchisor with 70 locations isn't a customer, it's a distribution event. One decision-maker, 70 boxes. Restaurant groups, rental management companies, marina operators. Highest leverage by far.
2. **Chambers of Commerce.** 300–800 members, wants to offer value, has no technology. Give them a live directory powered by member data; every member becomes a warm introduction. One deal is a city.
3. **Artists.** Free product, 30,000 QR impressions a week, and every code sits in a venue you want to sell.
4. **Agencies.** White label. Your trained reps become the agencies.
5. **Direct.** The first few hundred, by you, because platform flywheels never bootstrap.

Sales comp — where these programs die

- **Split commission on retention.** Half at install, half at 90 days retained. Pay full on signing and reps sell to anyone with a pulse.
- **Residual while the account lives.** Door-to-door people understand residuals. A rep with 60 accounts defends them and never leaves.
- **Let them earn on payment volume.** A rep who signed a rental manager in year one is still earning on \$3M of processing in year four. No other sales job can offer that.

The developer pitch

"You can vibe code this. You just need access."

LLMs made code free. Nobody can generate a Toast confirmation email, a phone with the owner's logins, real availability across four channels, or a business that said yes. Code stopped being scarce. Access didn't.

Keep the primitive set small. Five calls build the song request app:

<code>page.publish</code>	a public page with a QR
<code>payment.observe</code>	watch for a confirmation
<code>identity.match</code>	tie the payment to the request
<code>notify.send</code>	tell the artist
<code>data.write</code>	store it in their workspace

Same five build a tip jar, feedback form, waitlist, deposit collector, raffle, review request, table-side service call, lost-and-found, event RSVP. **That's the recruiting demo** — one page, five calls, nine apps.

ALTITUDE Verbs, not rows. Too high-level (`song.request`) and you're building apps, not a platform. Too low-level (raw email) and you've handed over the data. `payment.observe` returns "a payment arrived matching these criteria," never the inbox.

Build the app that can't exist anywhere else

"Plan my Saturday" → checks a charter's real availability, checks a restaurant's tables, books both, one confirmation. Two businesses, both consenting, neither sharing data with the other. No POS, OTA or directory can do it — they'd each need a partnership. You need two businesses that said yes.

Compounding, not accumulating

An app that only consumes capabilities adds one app. An app that *publishes* one makes every future app easier. Pay component authors from the apps built on them — your capability registry knows exactly which VAPP called which, which nobody else can measure. Publish the formula and the payouts; a verifiable split is a differentiator, a vague promise is a blog post.

10

Brand and narrative

The story

"Jobs took Unix, hid where it came from, and closed the ecosystem. I'm taking Linux, showing where it came from, and opening it."

NeXTSTEP was Mach plus BSD. Apple bought NeXT in 1997, it became Rhapsody, then Mac OS X. Every Mac and iPhone today runs on a foundation taken from Unix and covered up.

Most companies invent a narrative after the fact. You have a real one with forty years of lineage, and the logo carries it without explanation to anyone who knows.

The Ghost of Lisa

Lisa shipped January 1983 at \$9,995 — about \$32,000 today — with protected memory, cooperative multitasking, a document-centric interface, drag and drop, system-wide undo, the trash can, the menu bar, an integrated suite with a shared clipboard, and Gantt charts on a personal computer.

FOR THE AD **The Macintosh didn't get protected memory or real multitasking until Mac OS X in 2001.** Apple shipped it in 1983 then went backwards for eighteen years to hit a price point. The industry chose cheap-and-worse and took two decades to climb back.

~2,700 unsold Lisas were crushed and buried in a Logan, Utah landfill in 1989. In January 2023, on the fortieth anniversary, Apple released the Lisa source code through the Computer History Museum. **The machine they buried is now open source.**

Lisa	right, unaffordable, buried
Macintosh	affordable, closed
Ghost of Lisa	right, affordable, open

Lisa failed on access, not technology. You're not early — every component you need is mature and nearly free: a \$40 Android, a \$500 box with 128GB unified memory, models that run locally, forty years of Linux, Stripe. None of that existed five years ago. And naming v1 after the famous failure means nobody can accuse you of overpromising.

"It's not AI"

Everyone is slapping AI on everything at the exact moment small business owners stopped trusting it. "AI-powered" reads as *risk* to a 55-year-old charter captain.

"It's not AI. It does exactly what you'd do, and shows you proof it did it."

And it's literally true — the agent is deliberately dumb, reads a column, replays a recorded path, screenshots the result. Keeping AI out of the patent claims is right for a second reason: claims about a concrete remote-control method sit outside the hurricane of AI prior art.

The 1984 remake

Make the antagonist the **category**, not Apple. Big Brother is the gatekeeper — every AI company racing to own your computer and rent you access to your own business. Bigger target, more sympathetic, less exposure.

The line that does the work: **they want your business to live on their servers. Yours lives on your counter.**

Ending on the screen shutting off rather than the hammer is the better gesture — power through withdrawal rather than violence, and it maps to the product: the box is on your counter and the switch is yours.

Logo notes

- **Keep the cube.** Paul Rand's NeXT mark, and the cube is literally the product you ship. Most logos never get one idea that good.
- **Keep Tux** — it's the thesis, Jobs hid the Unix and you're showing the Linux — but **redraw it flat**. Glossy 3D cartoon against Rand-era geometry looks like two logos stapled together.
- **Build a standalone mark.** The cube alone, working at 32px, for favicon, app icon and the badge on the box lid. Rand's original reduces; yours needs the same discipline.
- **Replace the typeface.** The thin geometric sans is generic next to Rand's lettering. The box is industrial; the type should agree.
- **Buy every spelling** — anexgent, a-next-gent, anextagent, nextgent. Spoken it sounds exactly like "an agent," which is the joke and the discoverability problem. \$200 now, expensive later.

Stealth, resolved

Quiet about the mechanism, loud about the outcome. Sell "your hours update everywhere, your website shows real availability, your phone gets answered." No blog post about accessibility-service-driven remote control, no architecture diagram in the deck.

That resolves cleanly against open-sourcing, which otherwise contradicts stealth:

- **Open** — the platform, contracts, OS, SDK, capability spec. This recruits developers.
- **Not open** — the AppMap library, the fingerprint and repair engine, the per-vertical schemas. The accumulated operational knowledge the subscription buys.

Secrecy buys 6–18 months and costs every customer, developer and chamber deal in that window. Your real defenses — AppMap library, verified data, installed base, payment volume — all compound with speed, not silence. And your own content strategy guarantees the idea gets out anyway.

11

Competitive position

Why incumbents structurally cannot follow

- **Google** can't tell businesses to stop depending on Google Business Profile
- **Toast / Square** can't say "book direct, skip our fees"
- **FareHarbor / Booking.com** — commission is the company
- **Yext** can't stop charging for distribution; distribution is the product
- **Every seat-based SaaS** can't move to "it runs on your box" without torching ARR per customer

Not slowness — anti-correlation. Following you costs them their P&L.

REAL THREAT Not big tech. A well-funded fast follower with **no legacy revenue** — a team with \$5M and no conflicts. The answer is speed and accumulated data, which argues for 200 customers fast rather than staying quiet.

Omarchy — what it proves and what to avoid

DHH built roughly this product and deliberately declined to build this business. Arch + Hyprland + Quickshell, MIT, ~457 shell scripts, 249 theme files, a 51-chapter manual. Under the Omacom Foundation with ~\$18.5M in pledges. **No subscription, no hardware** — buyers get sent to Dell and Framework.

- **Steal:** the manual quality, theming as a product feature, the single-namespace CLI, and the agent-native positioning.
- **Avoid:** mutable Arch. 116 migration scripts are config-drift management — proof the state still lives on the customer's laptop. And `install/hardware/` is ~30 per-machine fix scripts, the infinite hardware matrix as a directory listing. Your certified SKUs delete that folder; read it as a components-to-avoid list.
- **Fatal for your model:** Arch is a rolling release. You cannot sell "supported until 2036" on top of it.

Home Assistant — the closest living template

Immutable appliance OS with A/B boot slots and RAUC OTA, shipped to millions of non-technical people. And the business is yours: Nabu Casa sells the hardware and a cloud subscription while the Open Home Foundation holds the project, everything Apache-2.0.

Somebody already ran your experiment and it worked. Read their update-system docs before committing to your own OS architecture.

Solid — the one that didn't work, and why

Berners-Lee's decentralized data project is your thesis exactly: personal data pods, apps request scoped permission, data stays with the person. From the guy who invented the web.

- **No killer app** — it asked people to move data for an abstract benefit. Nobody moves for principle.
- **No business model** — a standard looking for a company.
- **Too abstract** — "own your data pod" means nothing to a charter captain.

You have all three. Nobody has to understand decentralization to buy "my website shows what's actually open." That's the gap between a white paper and a company.

Capital — the conflict problem

A fund holding positions in POS, listings, booking, reputation or local search has a structural reason to soften what you do — not by blocking, but by slowing, redirecting, or "introducing" you to a partnership that neuters the direct-booking play.

- **Diligence the fund before the term sheet.** Ask directly what they hold in those categories. A straight answer is diagnostic either way.
- **Local money is strategically better, not just easier.** No portfolio conflict, they know the market, they're customers and references too, and they'll never tell you to go partner with Toast.
- **Don't sell control to solve a cash problem.** Revenue-based financing, customer prepayment, hardware deposits. At \$499/mo with real retention you can fund a lot off the business.
- **Information rights over board seats** for early money.

If the goal is genuinely that it outlives you and can't be captured, the structures exist — Open Home Foundation, Omacom Foundation, Linux Foundation projects, steward-ownership and purpose trusts. A verbal intention doesn't survive a term sheet; a trust deed does. But you can't decentralize something that doesn't exist — build it, get to self-funding, then put the governance around it from strength.

Traction before the call

- 20–30 **paying** businesses, not pilots
- Three months of retention — churn as a number, not a hope
- One ROI case with their statements — the charter that saved \$24K
- One vertical schema done, so customer 200 costs what customer 20 did

Six-month sprint, not a two-year build, and all of it in the first vertical. The thing that makes you fundable is the thing that makes you not need it.

12

The stack

Primary picks by layer. Everything here is open source and most of it was built to compose — Foreman with Katello with Pulp with Candlepin, Backstage with Keycloak, Sigstore with OCI registries. The full component list with alternates lives in the parts catalog.

LAYER	PRIMARY	ALSO ON THE LIST
OS image	bootc + ostree + greenboot + bootc-image-builder	rauc, Mender, Ubuntu Core, Omarchy (donor)
Node runtime	Podman + Quadlet, systemd	MicroShift for multi-service apps
Registry	Quay (cloud), zot (box)	Harbor
Content + channels	Pulp, Katello	Koji
Fleet	Foreman, flightctl (Edge Manager)	FleetDM, Argo CD
Entitlement	Candlepin	IBM License Service model for metering
Signing + trust	cosign, rekor , TUF	Skopeo, go-containerregistry
Supply chain	syft, grype, trivy, scancode, trufflehog, Conforma	CycloneDX / SPDX
Factory	Tekton, Buildah	Konflux
Identity	Zitadel or Keycloak	Backstage ships Keycloak integration
Authorization	SpiceDB or OpenFGA	OPA for contextual policy
Secrets / certs	OpenBao, cert-manager	
Data	Postgres, pgvector, SQLite + sqlite-vec	Meilisearch, OpenSearch, qdrant, Supabase
Events	NATS (box)	Kafka (cloud scale), Flink
Agent runtime	Hermes or BeeAI, behind your contract	OpenJarvis, QwenPaw, OpenClaw, Goose
Protocols	MCP + A2A	ACP is merged into A2A — don't invest
Gateway	IBM ContextForge	MCP Registry

LAYER	PRIMARY	ALSO ON THE LIST
Model routing	litellm, llama-swap	Llama Stack (build to Responses API)
Inference	llama.cpp (box), vLLM (bigger)	Ollama for dev, LocalAI, LLM Compressor
Models	Granite, Granite-Guardian	InstructLab, huggingface_hub, ROCm
Android	ADB, uiautomator2, scrapy, TextBee	Maestro + Studio (factory), trailblaze, appium-inspector, Artemis, mobile-mcp, AgentCPM-GUI, android_world
Browser	Playwright, playwright-mcp	browser-use, browser-harness, Stagehand, Steel, Browserbase
Linux desktop	AT-SPI	computer-use-linux, cua, ydotool
Workflow	Temporal, Event-Driven Ansible	Restate, Ansible, AWX, Node-RED
Integrations	Activepieces, Composio	n8n is not open source — Sustainable Use License
Voice	Wyoming, openWakeWord, silero-vad, faster-whisper, Kokoro	pipecat, LiveKit, Fonoster, hassil, speeches. Piper archived Oct 2025.
Files / data	Tika, Docling, langextract, magika	aifs, mem0 / letta, schema-dts
Sandboxing	Extism + wasmtime, gVisor	bubblewrap, Firecracker, microsandbox, OP-TEE
Verification	Playwright assertions, pixelmatch/odiff, ffprobe, qpdf	JHOVE
Packaging	Flatpak, PackageKit, AppStream	Skilla, xAI plugin-marketplace (donor), Backstage

LAYER	PRIMARY	ALSO ON THE LIST
Interfaces	Next.js, Flutter, cage (kiosk), Norigin (TV)	shadcn, Puck, JSON Forms, distrobox as the escape hatch
Networking	headscale / WireGuard	Tailscale, NetBird, SAM, Sunshine + Moonlight
Commerce	Stripe Connect, Lago	Kill Bill, dub
Backup / firmware	restic, Litestream, fwupd + LVFS	borg, rclone, snapper

What you own — nobody upstream does these

control-plane · identity / organizations / workspaces · capability-registry · ActionSpec · Constitution · policy-engine · credential-broker · agent-gateway · agent-contract and runtime adapter · model-router contract · execution-router · executor contracts (android, browser, linux, api, workflow, human) · AppMap spec and learn-once compiler · appmap-runtime · verification contract · evidence · Proof Ledger · VAPP manifest · marketplace semantics · certification definition · CyberCheck · Quantum Warehouse · exposure rules · device-manager · node-agent · updater and release-channel semantics · trust-service · recovery · sync · migration · operations · event-service · handoff-service · commerce-service · data-governance · edge-service · owner-ui · platform-admin · sdk · cli

RULE One job, one seam, one live implementation. Wrapper depth of exactly one — your contract calls the implementation directly. Anything marked ALTERNATIVE sits behind the same contract as the primary, never alongside it. Factory tools never ship. MCP and A2A are **projections** of your contracts, not layers under them.

13

IBM / Red Hat findings

Take the protocols and the models. Treat the "platforms" as reference designs. Refuse anything that hard-requires Kubernetes. The AI layer is overwhelmingly Apache-2.0 / MIT with real patent grants — much safer ground to build a Red-Hat-style business on than RHEL itself was.

COMPONENT	LICENSE	K8S?	VERDICT
BeeAI framework	Apache-2.0	no	Take — core agent runtime. LF AI & Data governed. RequirementAgent, tools, sandboxed code interpreter, memory, pause/resume, A2A adapters.
MCP + A2A	Apache-2.0 spec	no	Take — MCP is agent → tool (vertical), A2A is agent → agent (horizontal). ACP merged into A2A, Aug 2025 — legacy, don't invest.
ContextForge	Apache-2.0	no	Take — federates MCP + A2A + REST/gRPC. PyPI or plain Docker, air-gap capable, no telemetry.
Granite (all)	Apache-2.0	no	Take — published training-data disclosure and IP indemnification through Red Hat.
Granite-Guardian	Apache-2.0	no	Take — drop-in guardrail model, served on vLLM, called by your policy-engine.
Docling	MIT	no	Take — best-in-class document parsing, has an MCP server.

COMPONENT	LICENSE	K8S?	VERDICT
vLLM	Apache-2.0	no	Take — UC Berkeley origin, community governed, all major hardware backends, standalone container with an OpenAI-compatible API. Red Hat is the leading commercial contributor — this is literally the business you're copying.
LLM Compressor	Apache-2.0	no	Take — quantization for models you ship.
InstructLab	Apache-2.0	no	Take — laptop-capable fine-tuning and synthetic data. The customer-facing "improve it with your data" hook.
Llama Stack	MIT	no	Take as the API-server option. Build to the Responses API — the agents API is being deprecated.
KServe	Apache-2.0	yes	Reject for the box. "No Knative" ≠ "no Kubernetes."
llm-d	Apache-2.0	yes	Reject for the box — explicitly Kubernetes-native.
TrustyAI Guardrails Orchestration	Apache-2.0	yes	Every deployment path needs OpenShift cluster-admin. Run Granite-Guardian on vLLM instead — same detector models, none of the overhead.

COMPONENT	LICENSE	K8S?	VERDICT
OpenShift AI / Open Data Hub	mixed	yes	Reject. Mine the reference architecture (vLLM → Llama Stack → MCP → guardrails → RAG), reimplement with standalone containers.
watsonx Orchestrate	proprietary	—	Reject — the funnel. The ADK is a CLI into a paid platform; you need watsonx.ai or Orchestrate credentials to install Developer Edition. Copy the Agent Catalog concept for your App Store.

Watson is not one thing

Most of what IBM ships under that name is genuinely open and already in your list — Granite, Granite-Guardian, ContextForge, BeeAI, Docling, InstructLab, FMS-Guardrails, the watson-developer-cloud SDKs (Apache-2.0), watsonx-developer-hub, watsonx-data.

Commercial: **watsonx Orchestrate the platform**, watsonx.ai, watsonx Assistant.

The distinction that matters for your model: the watson-developer-cloud SDKs are Apache-2.0 *client libraries for paid cloud APIs* — open code, metered service behind it. Different from Granite or BeeAI, which run on your own box with nothing behind them.

Sandboxed execution — the one place the best tool isn't IBM/Red Hat

BeeAI ships a code interpreter, but the strongest self-hostable options are **microsandbox** (Apache-2.0, libkrun microVMs — libkrun is itself a Red Hat library — fully local, sub-200ms starts) and **E2B self-hosted** (Apache-2.0, Firecracker). MicroVM isolation is stronger than plain containers. Don't be steered to a weaker tool for brand consistency.

DEAD Daytona went closed source June 2026 — the former AGPL repo is archived. Don't build on it. Piper archived October 2025 — still in every tutorial, use Kokoro.

Seams and duplication to know about

- **Two agent runtimes, one orbit.** BeeAI (IBM) and Llama Stack (Meta + Red Hat) overlap. Pick one — framework-in-code or API-server — not both.
- **Two guardrail paths.** Granite-Guardian (model) vs TrustyAI Orchestrator (K8s service wrapping the same models). Same detectors, different packaging.
- **Two protocols that were three.** MCP, A2A, and ACP — now merged in.
- **BeeAI risk.** Pre-1.0 (0.1.x) with an explicit IBM notice that they may not maintain it. "Production-ready" in the README is marketing. Mitigated by Apache-2.0 + LF governance — you can fork and own it — but budget for that.

Licensing and trademark reality

- You can use vLLM, Granite, BeeAI freely. You cannot call your product "Red Hat" anything, and Red Hat's container images and RHEL bits carry subscription and trademark constraints — what CentOS/Rocky/Alma had to strip.
- Use **UBI or CentOS Stream** base images, not entitled RHEL images.
- Post-June-2023, CentOS Stream is the sole public repository for RHEL-related source; RHEL source proper is behind the paid Customer Portal and the subscription agreement discourages redistribution. This affects your OS base decision, not your AI stack.
- **Open in name only:** Red Hat AI Inference Server (support needs a subscription; the vLLM underneath is free), RHEL AI (Granite is Apache-2.0, the platform and indemnification are paid), Ansible Lightspeed (connect-service is self-hostable, the hosted model is commercial).

NOTE Treat the paid tiers as a blueprint for *your own* future product, not something to consume. vLLM → Red Hat AI Inference Server and Granite → RHEL AI are literal templates for "take OSS, harden, certify, indemnify, subscribe."

14

Modularity rules

One job → one seam → one live implementation. Wrapper depth of exactly one. Anything ALTERNATIVE sits behind the same contract as the primary, never alongside it. Factory tools

never ship. MCP and A2A are projections of your contracts, not layers under them.

The nine collisions found in the current stack

1. **Model routing is four shells deep.** `model-router` → `litellm` → `ollama` → `llama.cpp`.
Ollama is a wrapper over `llama.cpp` — two redundant hops. Go `model-router` → `litellm` → `{llama.cpp | vLLM}`; ollama stays on dev machines.
2. **Three ways to tap a button.** `mobile-mcp` wraps `uiautomator2`/`adb`, Maestro wraps `adb`, Artemis is a separate agent. One `AndroidExecutor` contract; **uiautomator2 is the only runtime implementation on the box**. Maestro, `trailblaze`, `appium-inspector` are factory-only. `mobile-mcp` is an MCP projection. Artemis is evaluated as a *replacement*, not an addition.
3. **Two agent runtimes.** Hermes and BeeAI do the same job. The contract makes the other swappable; running both makes neither.
4. **Two gateways.** `ContextForge` is *transport* (MCP/A2A/API federation). Agent Gateway is the *policy envelope*. Say it explicitly or they collide.
5. **Two identity systems.** Zitadel vs Keycloak. Tiebreaker: Backstage ships a Keycloak integration.
6. **Two authorization engines.** SpiceDB vs OpenFGA — both Zanzibar.
7. **Two event buses.** NATS on the box, Kafka only if cloud volume forces it. Never both on a node.
8. **Four search/vector stores.** `pgvector` for embeddings, Meilisearch for public text, `qdrant` stays out until vector load justifies a second store, `OpenSearch` is cloud log scale only.
9. **Three update-trust layers.** `bootc` is *delivery*, `cosign+rekor` is *signature* — different slots. TUF vs `cosign` genuinely overlaps; TUF earns its place only for key rotation and role delegation. `Mender` vs `bootc` is the same slot and has been open across three document generations.

The thirteen wires — everything plugs into one

Nothing gets forked and nothing gets picked. Everything below already speaks one of these, which is why they connect without integration work.

WIRE	WHAT SPEAKS IT
OpenAI-compatible HTTP	llama.cpp server, vLLM, ollama, LocalAI, llama-swap, litellm, Llama Stack, TGI — plus every cloud provider
MCP	ContextForge, playwright-mcp, mobile-mcp, docling-mcp, Composio, Activepieces, MCP Registry, your VAPPs, CyberCheck's public surface
A2A	A2A project, BeeAI (Agent Cards), Hermes, OpenJarvis, external agents, someone else's ChatGPT or Claude
ADB	uiautomator2, Maestro, scrapy, Appium, trailblaze, Artemis, mobile-mcp, DroidRun — all on the same connection to the same handset, simultaneously
CDP	Playwright, browser-use, browser-harness, Stagehand, Browserbase, Steel
AT-SPI	computer-use-linux, cua, ydotool, wtype
Wyoming	openWakeWord, silero-vad, faster-whisper, whisper.cpp, Kokoro, HA Assist, pipecat
OCI registry	Quay, zot, Harbor, skopeo, buildah, oras, cosign, rekor, bootc, Flatpak (OCI mode), your flow packs, your VAPPs
OIDC / OAuth2	Zitadel, Keycloak, Backstage, Cockpit, Grafana — run either or both, federated
Postgres wire	Postgres, pgvector, Supabase, Drizzle, Pulp, Candlepin, Keycloak, ContextForge, Temporal, Backstage
Events / CloudEvents	NATS, Kafka, Event-Driven Ansible, Temporal, Activepieces, Composio
Zanzibar check API	SpiceDB, OpenFGA — same check semantics
systemd + Quadlet	every container above, as a unit. Podman. No Kubernetes.

The eight seams

If a new repo doesn't plug into exactly one of these, it doesn't go in.

AgentRuntime	Hermes or BeeAI or OpenJarvis – one at a time
ModelRouter	litellm → llama.cpp / vLLM
Executor	android · browser · linux · api · workflow · human
Capability	your names and schemas; implementations are executors
Verifier	Playwright assertions, pixelmatch/odiff, ffprobe, qpdf
Policy	Constitution → SpiceDB → Zitadel
Package	VAPP manifest → wasmtime / Podman / Flatpak
Delivery	cosign → Quay/Pulp → Candlepin → bootc

NEVER LEAK The moment a VAPP manifest names Playwright or a resource-id, the executor stops being swappable and you're back to stacked shells. **Capabilities name *what*. Executors decide *how*.**

15

Open decisions

Each of these has stayed open across multiple document generations. Not choosing means building and maintaining two integrations forever — the contracts let you *swap* later, they don't let you run both for free.

DECISION	RECOMMENDATION	WHY
Mender vs bootc	bootc	Your own IBM assessment points the same way — Podman + quadlet, no Kubernetes. Mender's server-side fleet model fights the no-access posture.
Ubuntu Core	drop	Snaps add a second packaging system and Canonical's store in your update path.
Zitadel vs Keycloak	pick one	Backstage in → Keycloak (Red Hat ships the integration). Backstage out → Zitadel is lighter.
SpiceDB vs OpenFGA	pick one	Same Zanzibar model. V2 ledger says SpiceDB — close it there.
Hermes vs BeeAI	pick one	BeeAI is pre-1.0 with an IBM "may not maintain" notice. Your runtime contract makes either swappable — running both makes neither.
NATS vs Kafka	NATS on box	Kafka only if cloud volume forces it. Never both on a node.
TUF alongside cosign	only if needed	TUF earns its place for key rotation and role delegation cosign doesn't give. Real yes/no, not a both.
Temporal vs Restate	Temporal	Your own ledger already says evaluate, not run alongside.
Payments: own the rail?	own it for bookings you create	Or take zero transaction revenue and price the subscription higher. Don't try to invoice for money you never touch.

DECISION	RECOMMENDATION	WHY
Open core vs Red Hat model	Red Hat model	Nothing withheld. Open core creates a two-tier product and undercuts the credibility that's your strongest asset. Your moat isn't in a repository.

Two collisions worth naming because they're wrapper-on-wrapper, not choices:

- **ollama wraps llama.cpp.** `model-router` → `litellm` → `ollama` → `llama.cpp` is two redundant hops. Ollama is a dev tool; don't ship it on the box.
- **mobile-mcp wraps uiautomator2/adb.** It's an MCP projection of your executor, not a second executor. Maestro, trailblaze and appium-inspector are factory-only.

16

IP, credit and priority

Positioning and recordkeeping only. Scope, strength and timing of filings are counsel's call, not covered here.

Credit and protection are different goals with opposite tactics

Protection wants quiet until filings are locked. Credit wants loud, early and dated. You get both by splitting what from how — the same split as the stealth question, pointed at attribution instead of competition.

Publish the what

"A business's own phone, their own SIM, their own logins, operating the real apps they already use — and it shows you a screenshot proving it worked."

Keep the how quiet: the accessibility service architecture, the fingerprint gate, the repair loop, the flow format, the verification-on-a-different-path rule. Those are what take someone from "interesting" to "I can build that."

Dated priority — mechanisms

- **Cryptographic timestamping.** Hash the document, anchor the hash publicly. OpenTimestamps (free, Bitcoin-anchored) or rekor — already in your stack. Produces "this exact file existed on this date," unforgeable, un-backdateable. Ten minutes, costs nothing.
- **Signed git commits and tags.** A dated technical record in a repo you already have.
- **The videos.** Dated, public, you on camera — which is the plan anyway.
- **Third-party coverage.** A journalist writing about it on a date is evidence you didn't create.
- **The 2013 email chain.** Preserve properly — original headers, don't forward it around, archive somewhere with integrity.

COUNSEL Whether and when to publish detail interacts with filings in ways that vary by jurisdiction and timing. The recordkeeping above is a documentation question; the publishing decision isn't.

Demonstrations date better than descriptions

A written description of an idea is easy for someone to claim they had independently. A **dated video of it working on a real business account is not** — the artifact is too specific. Another argument for the thirty-second unbroken take being the first thing you make: it's marketing, proof of concept, and dated public record at once.

The honest perspective

Credit doesn't go to whoever said it first. It goes to whoever shipped it and got customers. Xerox PARC invented the GUI; Apple got the credit. Sun said "the network is the computer" a decade before anyone cared.

If someone announces phone-based remote control next year and you have 200 businesses running it plus a dated video from this month, you win the attribution fight without arguing it. With a blog post and no customers, you don't — regardless of who posted first.

CONCLUSION The strongest protection for credit is the same thing that's strongest for everything else: ship it.

Brand-mark exposure

Apple acquired NeXT in 1997 including the marks, and Paul Rand's cube is still owned IP. The cost of a dispute isn't legal fees — it's **sunk brand equity**: boxes badged, packaging printed, chamber deals signed under the name, a filing that gets opposed and blocks your own registration. And it arrives at whatever moment is worst — mid-raise, mid-production, mid-season.

Keep the brand cheap to change until you know. Run the homage where it's reversible — website, story, deck, videos. Hold the permanent, expensive surfaces — hardware badging, packaging, the filing — until counsel gives a read. Same provocation, none of the switching cost.

17

Founder assets

The things that are hard to acquire and easy to underuse.

DISTRIBUTION

Twenty years of sales

Car sales, door to door, trained reps out of rental operations. Small-business software dies on CAC because you can't afford a salesperson for a \$200/mo customer and they won't self-serve. A founder who can actually sell this buyer is the thing most teams in this category can't buy at any price.

PROOF

The 2013 email chain

Dated evidence you had the thesis before it was obvious. Converts "guy with an idea" into "guy who's been right and waiting." Deck opener: the email on screen with the date highlighted, then scroll into what you built.

NARRATIVE

The denial letters

Gatekeepers who said no, plus the MIT rejection. Framed on the wall is a photograph that ends up in an article. Free, and it can't be manufactured.

CHANNEL

On camera

Most people who can build this won't get in front of a lens; most people who make content aren't building anything. Being both is a genuine niche, and it's how a company with no ad budget gets distribution.

NETWORK

Local operators

30A and Destin relationships, and a path to a multi-unit franchisor. Money with no portfolio conflict, who are customers and references as well as capital, and who will never tell you to go partner with Toast.

METHOD

Playing dumb

Asking someone to explain what you already understand tells you what they know, how they think, and whether they're honest about their limits. People reveal more teaching than defending. Most useful tool you'll have interviewing CTOs.

How to position it

Be careful with "I know more than tech people." A strong engineer finds the gaps in ten minutes — everyone has them — and if you've claimed otherwise you've spent credibility you didn't need to spend. The stronger line, and the true one:

"I'm not an engineer. I know this business, this customer, and how every piece fits together better than anyone. I need someone who knows the systems better than I do."

Same rule with the prediction ledger: **specificity beats magnitude**. Three dated, checkable predictions land harder than "I predicted 90% of it." You have receipts — receipts beat claims, and you don't need the claim once you show the receipt.

Focus, resolved

"Focus on one thing" and "the pieces only work together" are both right — they're about different things.

- **The architecture has to be broad.** Email ingestion without execution is a parser. Execution without verification is a liability. That's design, not scope creep.
- **The go-to-market has to be narrow.** Amazon sold books. AWS launched with S3. Both had the whole thing in mind and neither led with it.

Build the platform, sell one sentence. You keep the entire vision; focus only has to happen in the sales call, which is the one place you're already better than anyone.

SELF-KNOWLEDGE The instinct to avoid the comfortable six figures is real and rare. But the risk doesn't go away when this works — it gets bigger, right after the first good quarter. What protects against it isn't discipline, it's obligation: employees whose rent depends on you, agencies with books of business, businesses that would be stranded. Build those in early and they'll hold you to it when you don't feel like being held.

18

Risks

App anti-automation detection

TEST WEEK ONE Some apps — banking especially, via Play Integrity — refuse to run with an active accessibility service. This is the only item that could be a hard wall rather than a hard problem, and it costs a weekend and one phone. It determines your addressable market.

Flow brittleness

REPLAY, THEN VERIFY Pure vision agents land 60–85% on novel multi-step tasks — fine for hours, unacceptable for invoices. Recorded deterministic paths plus mandatory end-state verification gets to ~98%, and a failed verification is a fleet signal rather than a customer complaint.

Scope

SHIP P2 BEFORE P4 Immutable OS, model orchestrator, GUI agent, data plane, marketplace, directory, hardware — each is a company. Only one sells. Software on the customer's own hardware buys revenue and evidence before you commit to inventory you can't return. And the documents have started auditing each other — 643 repos catalogued, no customer in any of them.

Email template churn

SAME MACHINE FareHarbor redesigns its confirmation and your parser breaks. Fingerprint the format, fail loudly rather than parse wrong, re-map once, push to everyone.

Phone as a wear item

CERTIFY, DON'T SHIP A random \$40 Android reintroduces the hardware matrix. Publish a supported device list — Moto G, Nokia/HMD, Samsung A-series, refurb Pixel. **Avoid Xiaomi MIUI, Oppo ColorOS, Vivo** — aggressive battery management silently kills the accessibility service, the single most likely failure mode in the design. And duty-cycle USB power in the cradle: charge to 80%, cut, drift to 60%. A phone pinned at 100% swells.

Overselling

NEVER CLAIM SMARTER Local models trail frontier models and customers notice in a week. Same mistake with the free app store — self-hosted alternatives are worse than the paid thing. Frame both as generosity, not replacement. Sell private, flat-fee, offline-capable, yours.

Single-vendor concentration

TWO VERTICALS EARLY If 80% of flows live inside one POS vendor, their roadmap is your roadmap. Same for payment volume — no vertical past ~35%.

Support cost

WHITE LABEL Consumers and small businesses generate 10–50× the tickets of enterprise admins. Agencies owning tier-1 is the structural answer; the approval queue and a one-button Recover are the product answers.

Data protection without a pile

CUSTOMER-HELD BACKUP A business running invoicing on your box will call you when a drive fails. Encrypted backup to a destination *they* choose — their NAS, their bucket — with a key you never hold. Protected, and you still can't read anything.

Fleet blindness

CERT RACK + OPT-IN RING No telemetry means no canary signal. Replace it with your own certification rack and a voluntary early-access ring. Don't pretend it's free.

The test that makes "decentralized" true

If you shut down tomorrow, does the box still work? Local data, local execution, last-known-good flows cached, no phone-home required to function. If yes, the claim holds and nobody else in the category can make it. If no, it's marketing and someone will find out.

Say it in the sales call — *"if I go out of business, your machine keeps working"* — it answers the biggest objection a small business has about buying from a company they've never heard of.

19

What to do next

This week

1. Buy one unlocked mid-range Android and a working SIM. One mini PC, 32GB+.
2. Install the real apps a Destin charter or restaurant uses. Turn on an accessibility service.
See what still runs. That answer determines everything else.
3. scrcpy mirroring, adb sending input. One afternoon.
4. Dump the UI tree as structured text.
5. Pick one flow — business hours in Google Business Profile. Record a human doing it.
6. Replay it. Screenshot the result. Read it back and confirm the hours changed.
7. Film it in one unbroken take, thirty seconds, no edits.

That video is the company

Your first ten customers, your first hire, your fundraiser. Someone says "update my hours" out loud and a real business's real listing changes on a real account, with proof. Nobody has to be told what it means.

Build it before you buy inventory, file a trademark, or design a screen.

Phases

	WHEN	GOAL	DONE WHEN
P0	Wk 1-4	Prove remote control on one flow	You say "update my hours to 9 to 5" and get a screenshot proving it happened.
P1	Wk 5-12	Flow library + approval board	Three businesses run five flows each, unattended, two straight weeks.
P2	Wk 13-20	Ship software, not hardware	Twenty-five businesses paying monthly, zero boxes shipped.
P3	Wk 21-32	The churn machine	A real app redesign detected, re-mapped and pushed fleet-wide inside 24 hours, no tickets.
P4	Wk 33-48	The appliance	100 units pass the certification suite, shipped to existing software customers first.
P5	Year 2	Widen	Model catalog, voice, regulated tier, multi-location fleet SKU where the customer holds the keys.

The weekly loop that is the subscription

Mon	fleet health – which verifications failed over the weekend
Tue-Wed	re-map broken flows, test on the certification rack
Thu	build and sign the flow pack, canary 5%, watch
Fri	staged rollout, changelog written for a restaurant owner
Ongoing	onboard new apps, add models, grow the shared library

If this loop runs, the business works. If it stops for a month, customers find broken flows before you do — and that's the only way this product dies.

Hiring the CTO

- **Not** a cloud/SaaS architect (wrong instincts — they'll rebuild what you're attacking), not an ML researcher (the agent is deliberately dumb), not a big-company CTO (they manage; you need hands on).
- **Yes:** a systems and infrastructure engineer who has **shipped physical product with software on it**. bootc, Yocto, Buildroot, Home Assistant OS. Fluent in Linux, containers, systemd, OTA. Thinks in assembly, not invention.
- **Where:** Home Assistant, Fedora IoT, Universal Blue and bootc contributor communities — they've already solved your OS problem for fun. Locally, Eglin and Hurlburt have a real concentration of embedded Linux engineers.
- **What makes it work:** their job is to say no, and yours is to accept it sometimes. A CTO who can't slow you down is useless; a founder who won't be slowed will churn through three of them.

The one sentence

"Your website shows what's actually available, right now, across every platform you're on — so people can book you direct instead of through the site taking twenty percent."

Every charter, marina, salon and rental operator on the Gulf Coast understands that immediately, and most of them are already angry about the commission. The architecture has to be broad; the sales call has to be narrow. You get to keep the whole vision — focus only has to happen in the one place you're already better than anyone.

20

Code and companions

What exists in the repo

Branch `claude/redhat-model-hardware-sales-iejnvo` on `the-ghost-ai`. Main is untouched.

<code>ghost/flow.py</code>	the script format – selectors not coordinates, <code>{{variables}}</code> , content-hashed identity
<code>ghost/runner.py</code>	fingerprint pre-flight → steps → read-back verify → run record with screenshots
<code>ghost/device.py</code>	adb: read the screen as nodes, tap, type, screenshot
<code>ghost/recorder.py</code>	map a flow by doing the task once
<code>ghost/pack.py</code>	bundle flows as OCI artifacts, cosign-signed, oras push/pull, refuses unverified installs
<code>platform/</code>	assembly map + Podman quadlets for the foundation spine (Postgres, Keycloak, zot, ContextForge)

Honest note on that code: `device.py` and `recorder.py` duplicate `uiautomator2` and Maestro Studio and should be replaced by them. What's worth keeping is the layer Maestro doesn't have – refuse-to-act gating, read-back as proof rather than assertion, the run record, and `pack/sign/push`. That thin layer is the owned contracts; everything under it is borrowed.

Flow format

```

flow: google_business.update_hours
version: 1
app: com.google.android.apps.town.businessmessaging
requires: [day, open, close]

fingerprint:                                # must be true before we touch anything
  - text: "Business hours"

steps:
  - tap: {text: "{{day}}"}
  - clear: {id: open_time}
  - type: {id: open_time, value: "{{open}}"}
  - tap: {text: "Save"}

verify:                                      # must be true after, read off the screen
  - contains: "{{open}}"
```

Verbs: `launch` · `stop` · `tap` · `type` · `clear` · `wait` · `sleep` · `back` · `home` · `scroll` · `assert`. Deliberately small — a flow anyone can read is a flow anyone can fix at 2am when an app redesigns.

Selectors: `text` (exact), `contains`, `id` (resource-id short name), `desc`, `cls`, `clickable`.

Companion documents

- **Build Plan** — phases P0–P5 with done-when gates, the weekly ops loop, economics, risk register, first thirty days.
- **Parts Catalog** — the full component list by layer with use / study / careful / skip verdicts on each, including licenses that matter for shipping hardware.

Working record, revision two — adds the six unsolved problems, IBM/Red Hat findings, modularity rules and the thirteen wires, IP and credit posture, founder assets, and the code reference.

Revision one notes. Phase durations assume a small team and will move; the dependency order should not. Vendor claims, pricing and product details should be re-verified against first-party sources before being used in legal, investment or public marketing contexts.